

SCPN Control: Compiling Stochastic Petri Nets into Spiking Neural Network Controllers for Real-Time Tokamak Plasma Control

Miroslav Sotek*
ANULUM CH & LI
(Dated: August 22, 2026)

We present SCPN-CONTROL, an open-source neuro-symbolic control engine that compiles Stochastic Petri Nets (SPNs) into spiking neural network (SNN) controllers with formal contract verification for tokamak plasma control. The compilation pipeline translates SPN graphs into leaky integrate-and-fire (LIF) neuron pools with stochastic bitstream encoding, enforcing pre/post-condition contracts on every control observation and action. The contribution is this compilation-and-contract architecture and a runtime-selectable controller stack (PID, nonlinear MPC, H_∞ , SNN) sharing a single contract interface, rather than raw throughput. The control compute is comfortably within the 4–10 kHz real-time band (native integrated control cycle 5.62 μ s P50 on the CI reference runner, reproducible; Section); a fielded loop is bounded by diagnostics, equilibrium reconstruction, and actuation rather than the controller, so we report the latency as meeting the budget with margin, not as a competitive speed claim. The package integrates a multi-layer Kuramoto–Sakaguchi phase dynamics engine driven by both the Paper 27 K_{nm} coupling matrix [1] and a plasma-native 8-layer K_{nm} whose entries encode experimentally grounded interactions (drift-wave/zonal-flow, NTM/bootstrap-current, ELM/pedestal), with a Unified Phase Dynamics Equation (UPDE) multi-layer solver monitored by a Lyapunov sliding-window stability guard. Five runtime-selectable controllers (PID, MPC, H_∞ , SNN, neuro-cybernetic) operate on a shared Grad-Shafranov equilibrium solver with L-mode and H-mode profile support. Disruption prediction with shattered pellet injection (SPI) mitigation, a Gymnasium-compatible environment, and a digital twin complete the control stack. The repository includes provenance-bound equilibrium fixtures and validation gates, but this manuscript does not promote them to independent experimental or same-case external-solver validation. The software is distributed under AGPL-3.0-or-later, with a separate commercial licence available, at <https://github.com/anulum/scpn-control>.

INTRODUCTION

Real-time plasma control in tokamaks demands sub-millisecond latency, formal safety guarantees, and runtime adaptability to regime transitions (L-mode to H-mode, ELM pacing, disruption avoidance). Existing plasma control systems (PCS) at DIII-D [2], ITER [3, 4], and EAST rely on classical PID or model-predictive control with hand-tuned gain schedules. These approaches lack formal verification of control actions and cannot adapt their control topology at runtime without operator intervention.

Recent work by Degraeve et al. [5] demonstrated deep reinforcement learning for magnetic control on TCV, achieving multi-objective shaping with a single neural network policy. While impressive, the approach treats the controller as a black box—no formal contract checking is applied to outputs before they reach the plant, and the policy cannot be inspected for reachability or liveness properties.

Spiking neural networks (SNNs) offer event-driven, low-latency computation that maps naturally to real-time control tasks [6, 7]. Stochastic Petri Nets (SPNs) provide a formal framework for modeling concurrent, stochastic processes with well-defined reachability, boundedness, and liveness properties [8, 9].

We bridge these paradigms. SCPN-CONTROL compiles an SPN control specification into an SNN controller, pre-

serving the formal contract structure of the Petri net as pre/post-condition checks on every control action. The SPN graph is declarative and inspectable; the compiled SNN is fast and neuromorphic-hardware-compatible.

Contributions.

1. A compilation pipeline from SPN graphs to LIF neuron pools with stochastic bitstream encoding, antithetic variance reduction, and formal contract verification (Section).
2. A multi-layer Kuramoto–Sakaguchi phase dynamics engine with the Paper 27 K_{nm} coupling matrix, a plasma-native 8-layer K_{nm} grounded in experimentally established interactions [10–13], UPDE multi-layer solver, PAC gating, and Lyapunov stability monitoring (Section).
3. Five runtime-selectable controllers (PID, MPC, H_∞ , SNN, neuro-cybernetic) sharing a common contract interface, with a Gymnasium-compatible environment for RL benchmarking (Section).
4. A CPU-only Rust-accelerated runtime with a measured 5.62 μ s P50 native integrated control cycle on the named CI reference runner; this is controller-side timing, not an end-to-end facility-loop claim (Section).
5. A Python package and five-crate Rust workspace

with configured coverage, documentation, security, and polyglot verification gates (Section).

RELATED WORK

Classical PCS. The DIII-D PCS [2] runs real-time equilibrium reconstruction (EFIT) at 4–10 kHz on dedicated hardware, coupled with PID shape controllers. ITER’s CODAC [3] specifies sub-millisecond plant I/O but relies on classical control algorithms. Walker and Humphreys [14] formalized vertical stability feedback but without runtime contract verification.

ML-based control. Degraeve et al. [5] trained a deep RL policy for multi-objective TCV shape control. The policy achieves real-time execution but provides no formal guarantees on output bounds. TORAX [15] provides a differentiable JAX-based transport simulator suitable for gradient-based optimization but does not include a control stack.

Spiking neural networks. Third-generation neural networks [7] have been proposed for neuromorphic control but not previously applied to tokamak plasma control. Stochastic computing with bitstream encoding [16] enables low-precision, fault-tolerant arithmetic suited to noisy plasma environments.

Petri nets in control. Petri nets have been used for supervisory control and discrete-event systems [9]. We study an SPN-to-SNN compilation path for continuous plasma-control research while making no priority claim over the broader neural, supervisory-control, or neuromorphic-control literature.

SPN TO SNN COMPILATION

Stochastic Petri Net Specification

A Stochastic Petri Net is a tuple $\mathcal{N} = (P, T, A, W, M_0, \Lambda)$ where P is a finite set of places, T a finite set of transitions, $A \subseteq (P \times T) \cup (T \times P)$ the arc set, $W : A \rightarrow \mathbb{N}$ the arc weights, $M_0 : P \rightarrow \mathbb{N}_0$ the initial marking, and $\Lambda : T \rightarrow \mathbb{R}_{>0}$ the stochastic firing rates.

Places represent physical observables (I_p , W_{MHD} , $\bar{n}_e$, vertical position Z) and controller states. Transitions encode control decisions (heating power, fueling, vertical stability corrections). The SPN is specified declaratively; the user does not write neural network code.

Compilation to LIF Neurons

The `FusionCompiler` maps each transition $t_j \in T$ to a stochastic leaky integrate-and-fire (LIF) neuron. The marking $M(p_i)$ of each place is encoded as a Bernoulli

Algorithm 1: SPN $\rightarrow$ SNN Compilation

Require: SPN $\mathcal{N} = (P, T, A, W, M_0, \Lambda)$, bitstream length L
 1: Build $\mathbf{W}_{\text{in}}$, $\mathbf{W}_{\text{out}}$ from arc set A and weights W
 2: **for** each transition $t_j \in T$ **do**
 3: Create LIF neuron with $V_{\text{th}} = \Lambda(t_j)$
 4: **end for**
 5: Pack $\mathbf{W}_{\text{in}}$ into `uint64` bitstreams (Eq. 1)
 6: Attach contracts from `ControlObservation`, `ControlAction`
 7: **return** `CompiledNet` ($\mathbf{W}_{\text{in}}$, $\mathbf{W}_{\text{out}}$, neurons, contracts)

Algorithm 2: Control Tick (Runtime)

Require: `CompiledNet`, observation $\mathbf{o}_t$, targets $\mathbf{r}$, marking $\mathbf{m}_t$
 1: **Pre-check:** verify $\mathbf{o}_t$ within physical bounds
 2: $\mathbf{e}_t \leftarrow (\mathbf{r} - \mathbf{o}_t)/\mathbf{s}$ $\triangleright$ normalized error
 3: Inject $\mathbf{e}_t$ into designated places
 4: $\mathbf{I} \leftarrow \mathbf{W}_{\text{in}} \mathbf{m}_t$ $\triangleright$ synaptic input
 5: $\mathbf{f}_t \leftarrow \text{LIF_step}(\mathbf{I})$ $\triangleright$ fire vector, Eq. 2
 6: $\mathbf{m}_{t+1} \leftarrow \text{Eq. 3}$
 7: $\mathbf{a}_t \leftarrow \text{decode}(\mathbf{m}_{t+1})$ $\triangleright$ push-pull + slew + clamp
 8: **Post-check:** verify $\mathbf{a}_t$ within actuator limits
 9: **return** $\mathbf{a}_t$, $\mathbf{m}_{t+1}$

bitstream of length L :

$$b_i^{(k)} \sim \text{Bernoulli}\left(\frac{M(p_i)}{M_{\text{max}}}\right), \quad k = 1, \dots, L \quad (1)$$

where $L \geq 64$ and M_{max} normalizes markings to $[0, 1]$. The bitstreams are packed into `uint64` words ($n_{\text{words}} = \lceil L/64 \rceil$) with per-element `splitmix64` seed derivation for statistical independence.

Arc weights $W(p_i, t_j)$ define two incidence matrices: $\mathbf{W}_{\text{in}} \in \mathbb{R}^{|T| \times |P|}$ (consumption) and $\mathbf{W}_{\text{out}} \in \mathbb{R}^{|P| \times |T|}$ (production). Each LIF neuron integrates:

$$V_j(t+1) = V_j(t) + \frac{\Delta t}{\tau_m} (V_{\text{rest}} - V_j(t)) + I_j(t) + \sigma \eta_j(t) \quad (2)$$

where $I_j = [\mathbf{W}_{\text{in}} \mathbf{m}]_j$ is the synaptic input from the current marking $\mathbf{m}$, $\tau_m = 10$ ms, and $\sigma = 0.1$ is the noise amplitude. When $V_j > V_{\text{th}}$, the neuron fires ($f_j = 1$) and resets to $V_{\text{reset}} = 0$.

The marking update rule preserves the Petri net semantics:

$$\mathbf{m}_{k+1} = \text{clip}(\mathbf{m}_k - \mathbf{W}_{\text{in}}^\top \mathbf{f}_k + \mathbf{W}_{\text{out}} \mathbf{f}_k, 0, 1) \quad (3)$$

Algorithm summarizes the compilation and Algorithm the runtime tick.

Stochastic Computing Acceleration

The dense forward pass $\mathbf{y} = \mathbf{W} \mathbf{x}$ can be computed via stochastic bitstream arithmetic: input bitstreams are

AND-ed with weight bitstreams, and the output is the popcount:

$$y_i = \frac{1}{L} \sum_{k=1}^L \text{popcount}(\mathbf{w}_i^{(k)} \& \mathbf{x}^{(k)}) \quad (4)$$

With `sc-neurocore` $\geq 3.8.0$, this can use `VectorizedSCLayer` with a Rust SIMD backend and packed `uint64` operations. Performance is reported only through committed, host-labelled benchmark artefacts.

Antithetic variance reduction. For n stochastic passes, we use antithetic sampling: for each base sample $u \sim \text{Uniform}(0, 1)$, we evaluate both $u < p$ and $u > 1 - p$, halving the variance of the firing probability estimator at no additional memory cost.

Formal Contract Verification

Every control tick verifies two contract layers:

Observation contracts. The observation $\mathbf{o}_t = (R_{\text{axis}}, Z_{\text{axis}})$ is checked against physical bounds. Errors are normalized and clipped to $[-1, 1]$ before injection into the SPN marking.

Action contracts. The decoded action $\mathbf{a}_t = (\Delta I_{\text{PF3}}, \Delta I_{\text{PF, tb}})$ is subject to slew-rate limiting ($|\dot{u}| \leq \dot{u}_{\text{max}}$) and absolute saturation ($|u| \leq u_{\text{abs}}$). Violations trigger a safe fallback (zero-action or last-known-good).

Physics invariants. Five runtime invariants are monitored: $q_{\text{min}} > 1$ (Kruskal–Shafranov [17]), $\beta_N < 2.8$ (Troyon limit [18]), Greenwald fraction < 1.2 [19], $T_i < 25$ keV (first-wall constraint), and $|\Delta W/W| < 0.01$ (energy conservation). Violations exceeding 20% of the threshold are classified as critical and trigger immediate protective action.

PHASE DYNAMICS ENGINE

Kuramoto–Sakaguchi with Global Field Driver

Following the Kuramoto model [20] with the Sakaguchi phase frustration extension [21] and SCPN Paper 27 [1], we model N coupled oscillators across $L = 16$ layers:

$$\dot{\theta}_n^{(\ell)} = \omega_n + \frac{1}{N_\ell} \sum_{m=1}^{N_\ell} K_{\ell\ell'} \sin(\theta_m^{(\ell)} - \theta_n^{(\ell)} - \alpha) + \zeta_\ell \sin(\Psi - \theta_n^{(\ell)}) \quad (5)$$

where ω_n are natural frequencies drawn from the Paper 27 canonical set, $K_{\ell\ell'}$ is the inter-layer coupling matrix, α is the Sakaguchi phase-lag parameter, ζ_ℓ is the per-layer global field coupling strength, and Ψ is the mean-field phase [22].

The order parameter measures synchronization:

$$R_\ell e^{i\Psi_\ell} = \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} e^{i\theta_n^{(\ell)}} \quad (6)$$

$R_\ell = 1$ indicates full synchronization; $R_\ell \rightarrow 0$ indicates incoherence.

Paper 27 K_{nm} Coupling Matrix

The 16×16 coupling matrix encodes inter-layer interactions with exponential distance decay and empirical calibration anchors:

$$K_{nm} = K_{\text{base}} e^{-\alpha_K |n-m|} + \sum_{(i,j) \in \mathcal{A}} c_{ij} \delta_{ni} \delta_{mj} \quad (7)$$

with $K_{\text{base}} = 0.45$, decay $\alpha_K = 0.3$, and calibration anchors $\mathcal{A}$:

(n, m)	(1, 2)	(2, 3)	(3, 4)
c_{nm}	0.302	0.201	0.252
(n, m)	(4, 5)	(1, 16)	(5, 7)
c_{nm}	0.154	≥ 0.05	≥ 0.15

The last two entries are cross-hierarchy boosts (L1↔L16 and L5↔L7) from Paper 27 §4.3.

Plasma–Native K_{nm} Coupling Matrix

The Paper 27 matrix was derived from a 16-layer model of consciousness dynamics. Tokamak plasmas have their own multi-scale hierarchy. We construct a dedicated 8×8 coupling matrix whose entries map directly onto experimentally established plasma physics interactions.

Layer hierarchy. Eight layers span the full range of tokamak timescales from micro-turbulence ($\sim \mu\text{s}$) to plasma–wall interaction ($\sim \text{s-min}$):

Layer	Process	Timescale
P0	Micro-turbulence (ITG/TEM)	μs
P1	Zonal flows ($E \times B$ shear)	ms
P2	MHD / tearing (NTM, 2/1 island)	ms–s
P3	Sawtooth / ELM cycles	10–100 ms
P4	Transport barrier (H-mode pedestal)	~ 100 ms
P5	Current profile ($q(\rho), l_i$)	s
P6	Global equilibrium (GS $\psi(R, Z)$)	s
P7	Plasma–wall interaction	s–min

Coupling construction. The backbone follows exponential distance decay as in Eq. (7), but with reduced base strength $K_{\text{base}} = 0.30$ and steeper decay $\alpha_K = 0.5$ to reflect the wider timescale separation in plasma:

$$K_{nm}^{\text{plasma}} = K_{\text{base}} e^{-\alpha_K |n-m|} + \sum_{(i,j) \in \mathcal{P}} p_{ij} \delta_{ni} \delta_{mj} \quad (8)$$

The physics overlays $\mathcal{P}$ encode seven experimentally grounded interactions:

(n, m)	p_{nm}	Reference
(0,1)	0.42	Drift-wave/zonal predator-prey [10]
(1,4)	0.28	$E \times B$ shear suppression [11]
(2,5)	0.35	NTM $\leftrightarrow$ bootstrap current [12]
(3,5)	0.30	Sawtooth $\leftrightarrow$ current redistribution [13]
(3,4)	0.32	ELM crash depletes pedestal
(4,6)	0.25	Transport $\leftrightarrow$ GS equilibrium [23]
(7,4)	0.20	PWI recycling modifies edge transport [24]

All couplings are symmetrised ($K_{nm} = K_{mn}$) and clamped non-negative after overlay application.

Instability mode bias. A mode parameter $m \in \{\text{baseline}, \text{elm}, \text{ntm}, \text{sawtooth}, \text{hybrid}\}$ multiplicatively amplifies couplings associated with the dominant instability. For example, **elm** mode scales K_{34} and K_{43} by $1.8\times$ (stronger pedestal-ELM interaction), while **hybrid** mode elevates all entries by $1.15\times$ to represent the tighter multi-scale coupling of advanced scenarios. This allows the phase engine to reflect the qualitative synchronization topology of a given operating regime without retuning individual entries.

Machine-parameter scaling. A configuration-driven builder adjusts K_{base} via a dimensionless beta proxy:

$$K_{\text{base}}(R_0, a, B_0, n_e) = 0.30 (1 + 0.5 \tilde{\beta}), \quad \tilde{\beta} = \frac{n_e a}{B_0^2} \quad (9)$$

where n_e is in units of 10^{19} m^{-3} , a in metres, and B_0 in Tesla. Higher beta implies more free energy driving instabilities, hence stronger multi-scale coupling. The cylindrical safety factor $q_{\text{cyl}} = 5a^2 B_0 / (R_0 I_p)$ is computed simultaneously; when $q_{\text{cyl}} < 1$ the builder automatically selects sawtooth mode, reflecting the physical inevitability of internal kink activity at low q .

Design for online adaptation. The static mode-switchable architecture is designed as the offline-verified foundation for a future real-time adaptive K_{nm} where diagnostic signals update coupling strengths online. Mirnov coil amplitudes would modulate K_{27} (MHD-PWI), Thomson scattering pedestal gradients would drive K_{34} (ELM-transport), and soft X-ray sawtooth inversion radius would scale K_{35} (sawtooth-current). The mode-switching interface already supports per-tick calls with $\sim 15 \mu\text{s}$ overhead; the remaining step is closing the loop from diagnostic pre-processing to coupling update, which we address in Section .

TABLE I. Lyapunov exponent λ vs. global field strength ζ ($N = 1000$, 200 steps, $\Delta t = 1 \text{ ms}$).

ζ	$\lambda (K = 0)$	$\lambda (K = 2)$
0.0	+0.01	+0.04
0.1	-0.03	-0.02
0.5	-0.23	-0.24
1.0	-0.49	-0.53
3.0	-1.65	-1.83
5.0	-3.01	-3.35

UPDE Multi-Layer Solver

The Unified Phase Dynamics Equation (UPDE) couples all 16 layers. For oscillator n in layer ℓ :

$$\begin{aligned} \dot{\theta}_n^{(\ell)} = & \omega_n^{(\ell)} + \frac{1}{N_\ell} \sum_m K_{\ell\ell} \sin(\theta_m^{(\ell)} - \theta_n^{(\ell)}) \\ & + \sum_{\ell' \neq \ell} K_{\ell\ell'} R_{\ell'} \sin(\Psi_{\ell'} - \theta_n^{(\ell)}) + \zeta_\ell \sin(\Psi - \theta_n^{(\ell)}) \end{aligned} \quad (10)$$

PAC gating. Optional phase-amplitude coupling modulates cross-layer drive based on source coherence:

$$\tilde{K}_{\ell\ell'} = K_{\ell\ell'} [1 + \gamma_{\text{PAC}} (1 - R_{\ell'})] \quad (11)$$

When source layer ℓ' desynchronizes ($R_{\ell'} \downarrow$), inter-layer coupling increases, implementing a stabilizing negative feedback. The repository benchmark harness measures the added compute path, but no portable PAC-overhead value is admitted in this manuscript.

Lyapunov Stability Guard

A sliding-window monitor tracks the Lyapunov candidate function [22]:

$$V(t) = \frac{1}{N} \sum_{n=1}^N (1 - \cos(\theta_n - \Psi)) \quad (12)$$

and computes the exponential rate over a configurable window of $W = 50$ samples:

$$\lambda = \frac{1}{\Delta t} \ln \left(\frac{V(t)}{V(t - W \Delta t)} \right) \quad (13)$$

$\lambda < 0$ indicates convergence toward synchronization. If $\lambda > 0$ for $K_{\text{max}} = 3$ consecutive windows, the guard triggers a coherence recovery action (increased ζ or controller gain boost).

Table I shows λ as a function of global field strength ζ for $N = 1000$ oscillators.

CONTROL ARCHITECTURE

Controller Suite

All five controllers share the contract interface of Section : observation pre-check, action post-check, and physics invariant monitoring.

PID. Dual-channel I_p + vertical position control with anti-windup and derivative low-pass filtering.

MPC. Gradient-descent optimization over a 10-step prediction horizon with L_2 regularization ($\lambda = 0.1$):

$$J = \sum_{t=0}^{N_p-1} \|\mathbf{A}\mathbf{x}_t + \mathbf{B}\mathbf{u}_t - \mathbf{x}_{\text{ref}}\|^2 + \lambda \|\mathbf{u}_t\|^2 \quad (14)$$

The plant model is a linearized 4-state system ($\mathbf{x} = [R_{\text{axis}}, Z_{\text{axis}}, R_x, Z_x]^\top$) with coil-current inputs. Action limits are ± 2.0 A per coil per step.

H_∞ . γ -iteration robust synthesis [25, 26] on a linearized vertical stability plant:

$$\dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}_1\mathbf{w} + \mathbf{B}_2\mathbf{u} \quad (15)$$

$$\mathbf{z} = \mathbf{C}_1\mathbf{x} + \mathbf{D}_{12}\mathbf{u} \quad (16)$$

The controller minimizes $\|\mathbf{T}_{zw}\|_\infty < \gamma$ via bisection on γ (range $[1.01, 10^6]$, $r_{\text{tol}} = 10^{-3}$, max 100 iterations). State-feedback and observer gains are obtained from the continuous algebraic Riccati equations; discrete-time adaptation uses zero-order-hold discretization. Robustness margin: $\pm 20\%$ multiplicative plant uncertainty.

SNN. Compiled from SPN specification (Section).

Neuro-cybernetic. Dual radial (R) + vertical (Z) spiking neuron pools ($n = 20$ neurons each) with push-pull decoding:

$$u = g \cdot \left(\frac{N_+}{N_w} - \frac{N_-}{N_w} \right) \quad (17)$$

where $N_\pm$ are spike counts in the positive/negative populations over a window of N_w ticks and g is the output gain.

Grad-Shafranov Equilibrium Solver

The GS equation in toroidal coordinates:

$$\Delta^*\psi \equiv R \frac{\partial}{\partial R} \left(\frac{1}{R} \frac{\partial \psi}{\partial R} \right) + \frac{\partial^2 \psi}{\partial Z^2} = -\mu_0 R^2 \frac{dp}{d\psi} - \frac{1}{2} \frac{dF^2}{d\psi} \quad (18)$$

is solved by Picard iteration on an $N_R \times N_Z$ grid (default 65×65).

Vacuum field. For each coil at (R_c, Z_c) with current I_c :

$$\psi_{\text{vac}} = \frac{\mu_0 I_c}{2\pi} \sqrt{(R + R_c)^2 + \Delta Z^2} \frac{(2 - k^2)K(k) - 2E(k)}{k^2} \quad (19)$$

where $k^2 = 4RR_c/[(R + R_c)^2 + \Delta Z^2]$ and K, E are complete elliptic integrals.

Profile modes. L-mode: $p(\rho) = p_0(1 - \rho^{\alpha_c})$. H-mode: core profile with mtanh pedestal transition at ρ_{ped} with configurable height and width.

Convergence to $\|\delta\psi\|_\infty < 10^{-6}$ typically in 12–18 iterations. Solver backends: Python (SOR, Picard), Rust (multigrid + SOR), or optional C++ HPC bridge.

Disruption Prediction and SPI Mitigation

An 11-dimensional feature vector is extracted from the plasma state at each tick: mean, std, max, and slope of the tearing mode signal, energy ($\langle s^2 \rangle$), last value, toroidal mode amplitudes ($n = 1, 2, 3$), toroidal asymmetry index, and radial spread.

The tearing mode physics follows the modified Rutherford equation [27]:

$$\frac{dw}{dt} = \Delta'(1 - w/w_{\text{sat}}) \quad (20)$$

Disruption is triggered when the island width exceeds $w > 8.0$ (mode lock).

An LSTM classifier (PyTorch, optional) predicts $P(\text{disrupt})$. When $P > 0.8$, shattered pellet injection (SPI) [28] is triggered with halo current and runaway electron post-disruption physics.

The predictor is trained on synthetic data derived from 11 reference DIII-D shot profiles [29] (6 anomalous: locked mode, density limit, VDE, tearing, beta limit; 5 safe: hybrid, H-mode, snowflake, negative delta, high beta).

Digital Twin and Gymnasium Environment

TokamakDigitalTwin. A 2D diffusive-reaction model on a 40×40 poloidal grid with core Gaussian heating ($+5.0$ K/step), configurable turbulent diffusivity ($D_{\text{turb}} = 0.5$ in island regions), and radiative losses ($\propto T^2$). Rational surface detection at $q = 1.5, 2.0, 2.5, 3.0$ marks regions of enhanced transport.

TokamakFlightSim. Isoflux flight simulator with first-order actuator dynamics ($\tau = 60$ ms), rate limits ($\leq 10^6$ A/s, ITER PF coil spec), sensor noise, and configurable measurement delay.

TokamakEnv. Gymnasium-compatible [30] environment wrapping the digital twin. Observation space: 4D continuous (R, Z, R_x, Z_x). Action space: continuous coil currents. Reward: $-\|\mathbf{x} - \mathbf{x}_{\text{ref}}\|^2$ with disruption penalty.

TABLE II. Control-loop latency on the recorded AMD EPYC 7763 CI runner. Isolated controllers use 200 warm-up steps and 2000 measured steps, except MPC with 500 measured steps. The integrated row uses 5000 steps in each of seven repeats.

Controller	Backend	P50 (μ s)	P99 (μ s)
PID	NumPy	0.42	0.52
SNN	NumPy	23.90	43.14
SNN	Rust	0.88	0.99
MPC ($N_p=10$)	acados	10679	11393
MPC ($N_p=10$)	osqp	16846	18090
MPC ($N_p=10$)	internal QP	18847	25172
H_∞	NumPy (general SS)	29.43	53.42
H_∞	Rust (2-state VDE)	1.39	1.49
Integrated control cycle	Rust native	5.62	6.11

BENCHMARKS AND VALIDATION

Control Loop Timing

Table II reports median (P50) and tail (P99) per-step latencies, measured by the committed reproducible benchmarks (warm-up plus repeated samples, not single-run averages) on the CI reference runner (AMD EPYC 7763). Every available backend is reported; a backend whose dependency is absent is listed as such rather than estimated. The repository publishes a separately measured Intel i5-11600K workstation column alongside the CI result, without treating either host as hardware-independent performance.

The SNN, H_∞ , PID, and native integrated control cycle all sit well within the 4–10 kHz real-time band on the control-compute side. This is a necessary-not-sufficient property: a fielded loop is bounded by diagnostics, equilibrium reconstruction, and actuation, not by these control-compute figures, so they are reported as meeting the budget rather than as an end-to-end speed advantage over fielded systems. The nonlinear MPC is millisecond-scale on every QP backend, with **acados** the fastest (≈ 10.7 ms); the full five-backend comparison (internal, scipy, osqp, casadi, acados) is regenerated by the repository benchmark. The microsecond MPC regime is not reachable through the Python **step.rti** path with any QP backend, because per-stage set/get marshalling across the Python–C boundary dominates the tick; a microsecond MPC tick would require a non-Python (C/Rust) control loop, not merely a compiled QP solver. The MPC row is therefore reported as a research-grade tick latency, not a real-time claim.

Capability Context

The related systems solve different problem slices. DIII-D PCS demonstrates fielded equilibrium reconstruc-

tion and control [2]; TORAX provides differentiable transport [15]; FreeGS provides an open Grad–Shafranov solver [31]; and the TCV work demonstrates learned magnetic control [5]. The contribution evaluated here is the repository’s explicit SPN-to-SNN compilation and contract boundary. No overall superiority claim follows from this capability difference.

Kuramoto Phase Sync: Python vs. Rust
The repository includes a side-by-side Python/Rust benchmark over increasing oscillator counts. Those observations are useful for local regression, but this paper does not reproduce them as a portable speedup claim because the legacy table is not accompanied by a sealed raw host-context artefact at the manuscript revision.

Equilibrium RMSE

The repository carries checksum-bound GEQDSK fixtures and regression gates. These are software-regression evidence, not independent reconstruction accuracy against a current experimental reference code on identical measurements. The legacy percentage table is therefore withdrawn from this review draft pending a same-case, provenance-complete comparison.

Phase Dynamics Convergence

The repository provides deterministic phase-synchronisation scenarios and a sliding-window Lyapunov monitor. Their outputs establish reproducible simulated behaviour for the configured model only; they do not establish experimental plasma stability or an independent fail-closed safety interlock.

SOFTWARE ARCHITECTURE

Python Package

The Python package is organised by responsibility into **scpn/**, **core/**, **control/**, and **phase/**, with validation and public interfaces outside those implementation packages. Exact module and test inventories are generated from the repository tree rather than frozen in the manuscript because they change as the software evolves.

Rust Workspace

Five crates compiled via PyO3 [32]:

1. **control-types:** `PlasmaState`, `EquilibriumConfig`, `ControlAction`, physical constants.
2. **control-math** (14 modules): multigrid, SOR, GMRES, Chebyshev, FFT, Boris pusher, LIF neuron, Kuramoto step, symplectic integrator, tridiagonal solver.
3. **control-core** (20 modules): GS kernel, transport, vacuum field, X-point detection, inverse solver, pedestal, stability, RF heating, VMEC/BOUT++ interfaces.
4. **control-control** (10 modules): PID, MPC, H_∞ , SNN pool, SPI mitigation, digital twin, optimal control, SOC learning.
5. **control-python:** PyO3 bindings exporting `PyFusionKernel`, `PySnnPool`, `PyMpcController`, `PyPlasma2D`, `PyTransportSolver`, `PyRealtimeMonitor`, and `SCPN` kernels (`dense_activations`, `marking_update`, `sample_firing`).

Python code auto-detects the Rust backend at import time via `importlib.util.find_spec` and dispatches hot-path calls accordingly, with a pure-Python fallback for all functionality.

Testing and CI

The repository runs module-specific Python tests, Rust workspace tests, and integration checks across its hosted workflows. The configured Python package coverage gate is 100%; the manuscript does not substitute a historical test or workflow count for exact-head CI evidence:

- **Python:** `pytest`, `ruff`, `mypy`, `bandit`, coverage (100% configured package gate), scientific regression gates
- **Rust:** `cargo test`, `clippy`, `fmt`, `audit`, Criterion benchmarks (Kuramoto, transport, LIF, Boris)
- **Integration:** named public/runtime paths, documentation builds, and generated-evidence drift checks

LIMITATIONS

We state limitations explicitly so they are not overstated by inference:

1. **Equilibrium:** fixed- and free-boundary implementations are repository software surfaces; independent same-case reconstruction against a current external reference and facility diagnostics remains open.
2. **Transport:** native and adapter surfaces do not establish quantitative equivalence to TGLF, GENE, GS2, CGYRO, or QuaLiKiz without admitted external artefacts on identical inputs. Nonlinear CBC saturation and unit-normalised cross-code agreement remain open.
3. **Validation:** repository fixtures include synthetic and public reference data. Offline replay is not live MDSplus acquisition or a facility experiment, and no fielded PCS validation is claimed.
4. **Disruption prediction:** available measured-shot evidence is offline and dataset-bounded; it is not a deployed predictor or actuator authority.
5. **Deployment:** CODAC, EPICS, IMAS, HDL-export, and hardware-facing adapters are integration surfaces, not proof of installation, post-route timing, HIL operation, or certified plant control.
6. **Plasma K_{nm} calibration:** Coupling strengths are order-of-magnitude estimates derived from published scalings, not fitted to experimental multi-diagnostic correlation data. Real-time adaptive mode requires closing the diagnostic-to-coupling loop.
7. **Rust acceleration:** optional acceleration is reported only where paired, host-labelled artefacts exist. Performance does not imply mathematical parity outside the explicitly tested path.

CONCLUSION AND FUTURE WORK

We presented a neuro-symbolic approach to tokamak plasma control that compiles Stochastic Petri Nets into spiking neural network controllers through a formal pipeline preserving contract verification at every control tick.

The technical result is an inspectable compilation-and-contract architecture: control logic is specified as a Petri-net graph, compiled into LIF neuron pools, and checked at the observation/action boundary. The committed host-labelled benchmarks show microsecond-scale controller-side native handoff on their named machines, but do not compare end-to-end facility loops. The phase engine adds a plasma-native 8-layer K_{nm} grounded in drift-wave/zonal-flow [10], NTM/bootstrap-current [12], and ELM/pedestal interactions, with machine-parameter

scaling via $\tilde{\beta}$ and automatic sawtooth-mode selection at $q_{\text{cyl}} < 1$.

Future work targets:

- **Real-time adaptive K_{nm} :** closing the loop from diagnostic pre-processing (Mirnov coils, Thomson scattering, soft X-ray) to per-tick coupling updates, making $K_{nm}(t)$ a function of the instantaneous plasma state rather than a static operating-point selection
- Same-case independent equilibrium and transport-code comparison
- Shot-disjoint measured-data validation with uncertainty and failure accounting
- Named-target post-route and hardware-in-the-loop evidence
- Facility-approved non-actuating shadow operation

The software is available at <https://github.com/anulum/scpn-control> under AGPL-3.0-or-later, with a separate commercial licence available.

This work builds on the SCPN theoretical framework (Papers 1–27) and the `scpn-fusion-core` codebase. We thank the open-source fusion community for reference equilibria, the DIII-D team for published GEQDSK data, and the Commonwealth Fusion Systems team for SPARC equilibrium parameters.

* protoscience@anulum.li

- [1] M. Sotek, The K_{nm} matrix: A simulation framework for modelling multi-scale bidirectional causality (2026), sCPN Paper 27, https://www.academia.edu/143833534/27_SCPN_The_Knm_Matrix.
- [2] J. R. Ferron, M. L. Walker, L. L. Lao, H. E. St John, D. A. Humphreys, and J. A. Leuer, Nucl. Fusion **38**, 1055 (1998).
- [3] ITER Organization, *CODAC Core System Overview*, Tech. Rep. (ITER Organization, 2020) iTER_D-2V3V8G.
- [4] D. A. Humphreys, T. A. Casper, N. Eidietis, M. Ferrara, D. A. Gates, I. H. Hutchinson, G. L. Jackson, E. Kolemen, J. A. Leuer, J. Lister, *et al.*, Phys. Plasmas **22**, 021806 (2015).
- [5] J. Degraeve, F. Felici, J. Buchli, M. Neunert, B. Tracey, F. Carpanese, T. Ewalds, R. Hafner, A. Abdolmaleki, D. de las Casas, *et al.*, Nature **602**, 414 (2022).
- [6] A. Tavanaei, M. Ghodrati, S. R. Kheradpisheh, T. Masquelier, and A. Maida, Neural Networks **111**, 47 (2019).
- [7] W. Maass, Neural Networks **10**, 1659 (1997).
- [8] M. A. Marsan, G. Balbo, G. Conte, S. Donatelli, and G. Franceschini, *Modelling with Generalized Stochastic Petri Nets* (Wiley, 1995).
- [9] T. Murata, Proc. IEEE **77**, 541 (1989).
- [10] P. H. Diamond, S.-I. Itoh, K. Itoh, and T. S. Hahm, Plasma Phys. Control. Fusion **47**, R35 (2005).
- [11] P. W. Terry, Rev. Mod. Phys. **72**, 109 (2000).
- [12] R. J. La Haye, Phys. Plasmas **13**, 055501 (2006).
- [13] F. Porcelli, D. Boucher, and M. N. Rosenbluth, Plasma Phys. Control. Fusion **38**, 2163 (1996).
- [14] M. L. Walker and D. A. Humphreys, Automatica **45**, 665 (2009).
- [15] A. Fanni, J. Citrin, F. Felici, *et al.*, Nucl. Fusion (2024), arXiv:2406.06718.
- [16] A. Alaghi and J. P. Hayes, ACM Trans. Embed. Comput. Syst. **12**, 1 (2013).
- [17] J. P. Freidberg, *Ideal MHD* (Cambridge University Press, 2014).
- [18] F. Troyon, R. Gruber, H. Saurenmann, S. Semenzato, and S. Succi, Plasma Phys. Control. Fusion **26**, 209 (1984).
- [19] M. Greenwald, Plasma Phys. Control. Fusion **44**, R27 (2002).
- [20] Y. Kuramoto, Lecture Notes in Physics **39**, 420 (1975).
- [21] H. Sakaguchi and Y. Kuramoto, Prog. Theor. Phys. **76**, 576 (1986).
- [22] J. A. Acebrón, L. L. Bonilla, C. J. Pérez Vicente, F. Ritort, and R. Spigler, Rev. Mod. Phys. **77**, 137 (2005).
- [23] H. Lütjens, A. Bondeson, and O. Sauter, Comput. Phys. Commun. **97**, 219 (1996).
- [24] P. C. Stangeby, *The Plasma Boundary of Magnetic Fusion Devices* (Institute of Physics Publishing, 2000).
- [25] K. Zhou, J. C. Doyle, and K. Glover, *Robust and Optimal Control* (Prentice Hall, 1996).
- [26] K. Glover and J. C. Doyle, Syst. Control Lett. **11**, 167 (1988).
- [27] P. C. de Vries, M. F. Johnson, B. Alper, P. Buratti, T. C. Hender, H. R. Koslowski, and V. Riccardo, Nucl. Fusion **51**, 053018 (2011).
- [28] M. Lehnert, K. Aleynikova, P. B. Aleynikov, D. J. Campbell, P. Drewelow, N. W. Eidietis, Y. Gasber, R. S. Granetz, Y. Gribov, N. Hartmann, *et al.*, J. Nucl. Mater. **463**, 39 (2015).
- [29] C. Rea and R. S. Granetz, Fusion Sci. Technol. **76**, 912 (2019).
- [30] M. Towers, J. K. Terry, A. Kwiatkowski, J. U. Balis, G. de Cola, T. Deleu, M. Goulão, A. Kallinteris, A. KG, M. Krimmel, *et al.*, Gymnasium (2023), <https://github.com/Farama-Foundation/Gymnasium>.
- [31] B. Dudson *et al.*, FreeGS: Free-boundary Grad-Shafranov solver (2023), <https://github.com/freeds-plasma/freeds>.
- [32] PyO3 Contributors, PyO3: Rust bindings for Python (2024), <https://pyo3.rs>.